

Tutorial: PYNQ DMA (Part 1: Hardware design)

[cathalmccabe](#) 1 October 13, 2021, 1:29pm

PYNQ DMA tutorial (Part 1: Hardware design)

This tutorial will show you how to use the Xilinx AXI DMA with PYNQ. It will cover adding the AXI DMA to a new Vivado hardware design and show how the DMA can be controlled from PYNQ. This tutorial is based on the v2.6 PYNQ image and will use Vivado 2020.1. If you are using a different PYNQ version you should be able to follow the same steps in this tutorial, but you should make sure you are using the supported version of Vivado for that PYNQ release.

The PYNQ-Z2 board was used to test this design.

The source files to rebuild this example and the compile .bit and .hwh can be found [here](#)

Create a new Vivado project

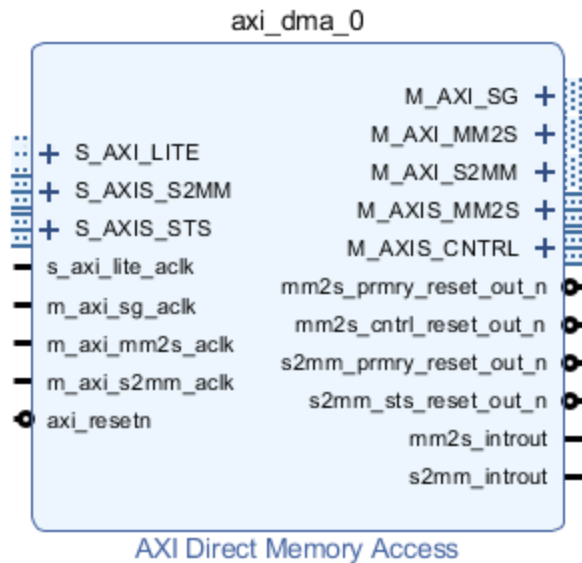
This tutorial will create a design for the PYNQ-Z2 (Zynq) board. You should be able to follow the instructions to create a similar design for other Zynq or Zynq Ultrascale+ boards. The steps for other Zynq boards should be the same. There will be modifications for Zynq Ultrascale+ (E.g. PS settings) that won't be covered now, but you can ask questions in the comments or post a new question to the support forum. .

The first step for every Zynq hardware design is to add and configure the PS block. Rather than repeat instructions [Create a hardware design](#). If you are new to Vivado and creating Zynq designs, I would recommend you at least read through all of this tutorial.

Follow the steps in a previous tutorial: [Create a hardware design](#) to create a Vivado project, and add and configure the Zynq PS block with default settings. Stop just before the section on *Adding blocks to your design* and continue with this tutorial:

Adding the DMA

In the Vivado block diagram which should contain the Zynq PS block, add the **AXI Direct Memory Access** block to your design. You should see the following IP block. You see the ports for the default configuration of this IP.



The name of the AXI IP will be visible later from Python, so it is good practice to rename IP blocks from the default, even if it is only to remove the training “_0”

- Select the block and rename it to `dma`

DMA background

The DMA allows you to stream data from memory, PS DRAM in this case, to an AXI stream interface. This is called the *READ channel* of the DMA. The DMA can also receive data from an AXI stream and write it back to PS DRAM. This is the *WRITE channel*.

The DMA has AXI Master ports for the read channel, and another for the write channel, and are also referred to as *memory-mapped* ports - they can access the PS memory. The ports are labelled MM2S (Memory-Mapped to Stream) and S2MM (Stream to Memory-Mapped). You can consider these as the read or write ports to the DRAM for now.

Control port

The DMA has an AXI lite control port. This is used to write instructions to configure, start and stop the DMA, and readback status.

AXI Masters

There are two AXI Master ports that will be connected to the DRAM. `M_AXI_MM2S` (READ channel) and `M_AXI_S2MM` (WRITE channel). AXI masters can read and write the memory. In this design they will be connected to the Zynq HP (High Performance) AXI Slave ports. The width of the HP ports can be changed in the Vivado design, however these ports are configured when PYNQ boots the board. You need to make sure the width of the ports in your Vivado design matches the PYNQ boot settings. In all official PYNQ images, the width of the HP ports is 64-bit.

If you set the HP ports to 32-bit if your design by mistake, you will likely see only 32-bits out of every 64-bits are transferred correctly.

AXI Streams

There are two AXI stream ports from the DMA. One is an AXI master Stream (M_AXIS_MM2S) and corresponds to the *READ* channel. Data will be read from memory through the M_AXI_MM2S port and sent to the M_AXIS_MM2S port (and on to the IP connected to this port).

The other AXI stream port is an AXI Slave (S_AXIS_S2MM). This is connected to your IP. The DMA receives AXI stream data from the IP, and writes it back to memory through the M_AXI_S2MM port.

If the IP is not ready to receive data from the M_AXIS port, then this port will stall. You can also use Stream FIFOs. If the IP tries to write back data but a DMA *write* has not started, the S_AXIS channel stall the IP. Again, FIFOs can be used if required. The DMA has some built in buffering so if you are trying to debug your design you may see some (or all) data is read from memory, but it may not necessarily have been sent to your IP and may be queued internally or in the HP port FIFOs.

Scatter gather support

PYNQ doesn't support scatter gather functionality of the DMA. This is where data can be transferred from fragmented or disjointed memory locations. PYNQ only supports DMA from contiguous memory buffers.

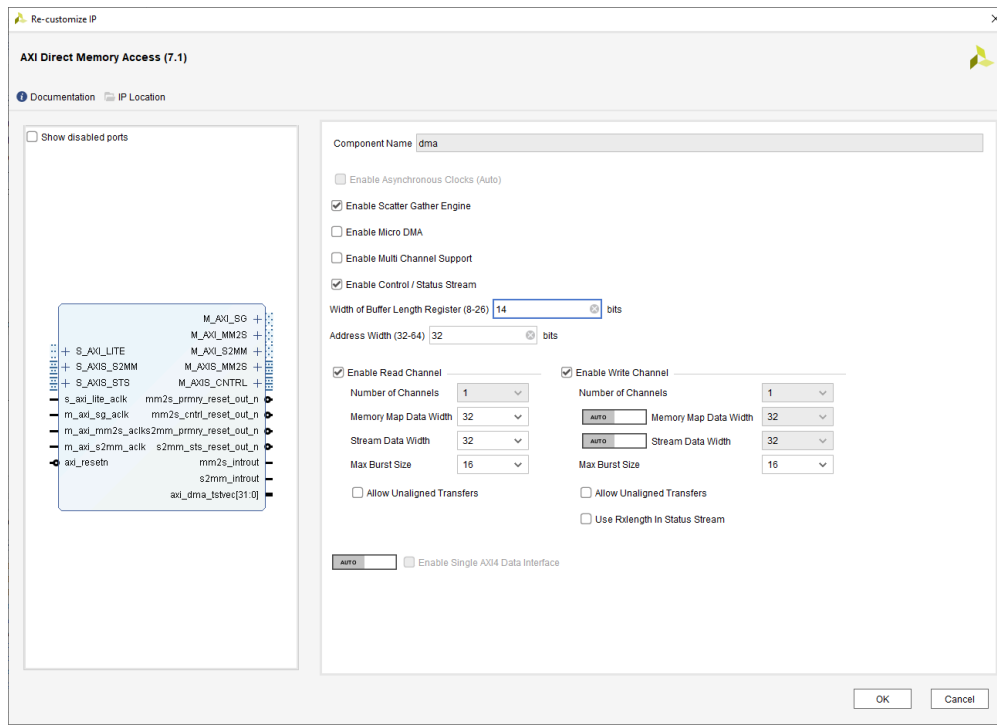
Scatter-Gather can be enabled on the DMA to allow multiple transfers of up to 8,388,608 bytes (from contiguous memory buffers). If you do this, you need to use the SG M_AXI ports instead of the M_AXI ports. This is not covered in this tutorial.

An alternative to SG for large transfers is to segment your memory transfers in software into chunks of 67,108,863 or less and run multiple DMA transfers.

Configure the DMA

- Double click the DMA to open the configuration settings

You should see the following default settings:



- Uncheck *Enable Scatter Gather Engine* to disable Scatter Gather

Buffer Length Register

- Set the *Width of Buffer Length Register* to 26

This value determines the maximum packet size for a single DMA transfer. **width = 26** allows transfers of 67,108,863 bytes - the maximum size the DMA supports. I usually set this value to the maximum value of 26. If you know you will never need more than a smaller size transfer, you can set this to a smaller value and save a small amount of PL resources. I prefer to set the maximum value for flexibility as the hardware resource increase is relatively modest.

When using the DMA if you try to do a transfer but only see that the first part of your buffer is transferred, check this value in your hardware design and check how much data you are transferring. Leaving the default with set to 14-bits is a common mistake which will limit the DMA to 16,384 byte transfers. If you try to send more than this the transfer will terminate once the maximum number of bytes supported is transferred. Remember to check the size of the transfer in *bytes*.

Address width

- Check the address width is set to 32. In this example, I will connect the DMA to the PS memory which has a 32-bit for Zynq. You can set this up to 64-bit if you are connecting this to a larger memory, for example if you are using a Zynq Ultrascale+ or if your DMA is connected to a PL connected memory.

DMA read and write channels

This example will use both the read and write channels of the DMA, but you may only need to enable one or both of these channels. You could also have multiple DMAs in your design.

- For this design, leave both both *read* and *write* channels enabled
- Set the memory mapped data width to 64 match the HP port (defined in the PYNQ image and applied at boot time)

You can leave the write channel set to **auto** but you should check later that this gets updated to 64.

- Set the stream data with to match your IP stream width. In this example I will leave it set to 32.

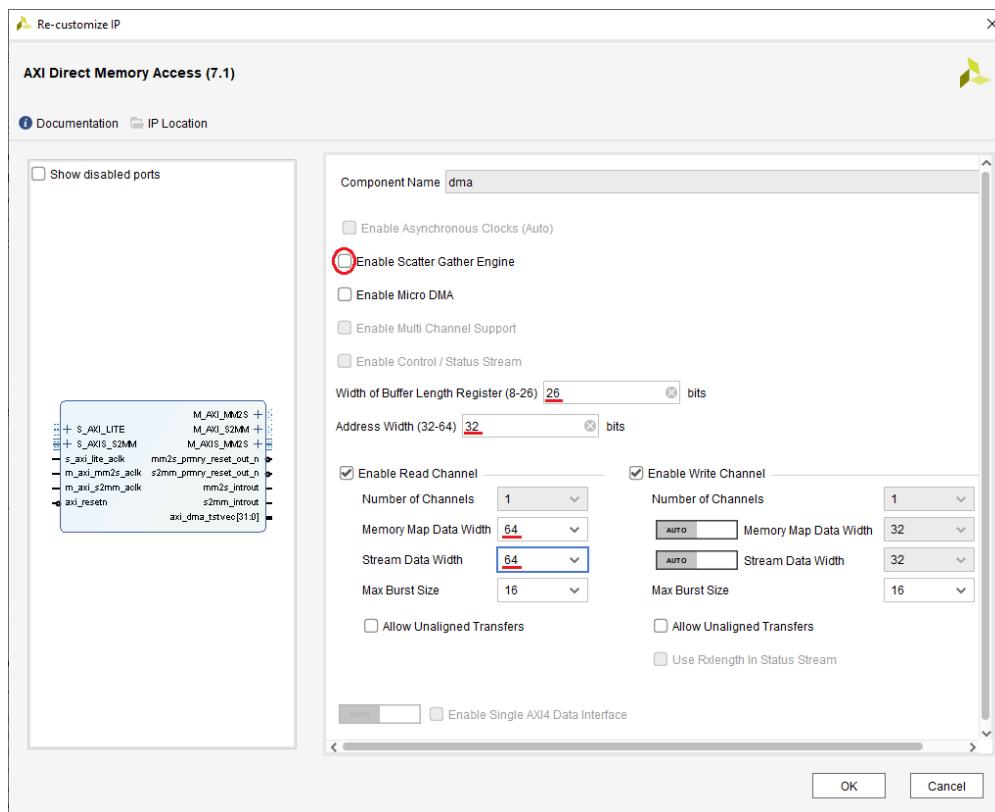
You can set different data widths and Vivado should add AXI interconnect to automatically step up or down your data widths, or give a warning if there is a mismatch in your design. For best results - efficient hardware implementation, and data flow, match the settings of the DMA to your datapath.

You can increase the burst width to improve efficiency of your data transfers. There is usually a small hardware resource utilization will increase as you increase the max burst size, but this should not have significant impact of the overall utilization.

- Make sure *Allow unaligned transfers* is **NOT** enabled.

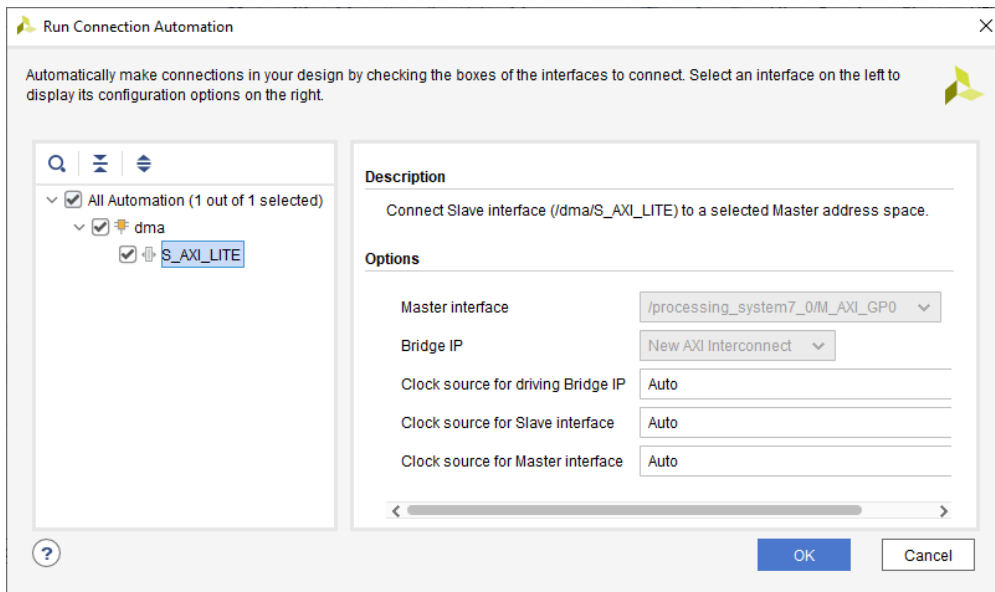
This is not supported with PYNQ.

- Click OK to accept the changes.



Connect the DMA

- Click on *Run connection automation* to open the dialog box
- Check the **S_AXI_LITE** box under the dma and click OK

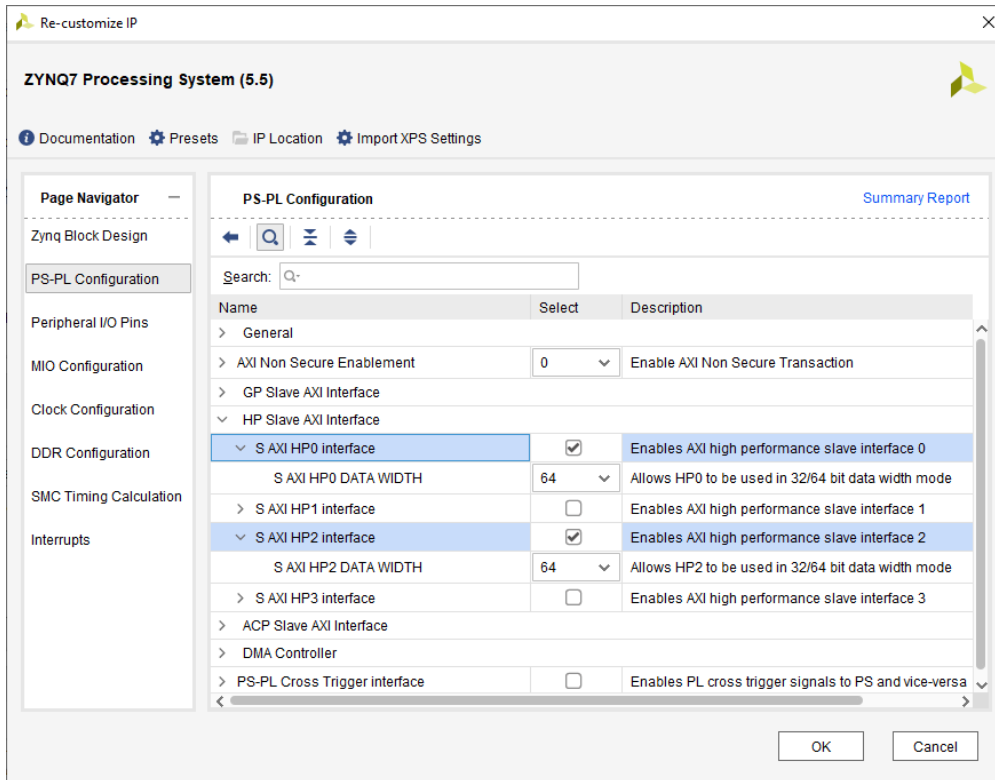


This connects the S_AXI_LITE port of the DMA to the Zynq PS M_AXI_GP0 port. This is the control port for the DMA.

Memory mapped connections

The DMA AXI master ports need to be connected to the PS DRAM. This will be done through the Zynq HP (AXI Slave) ports. These ports are not enabled by default. Internally there are two connections to the PS memory that the four HP ports are connected to. HP0 and HP1 share a switch to one port, and HP2 and HP3 share a switch to the other. The difference may not be noticeable for this example and some design, but when only two HP ports are required, it is more efficient to connect them to HP ports that don't share a switch. i.e. HP 0 and HP 2 or HP 1 and HP 3 together.

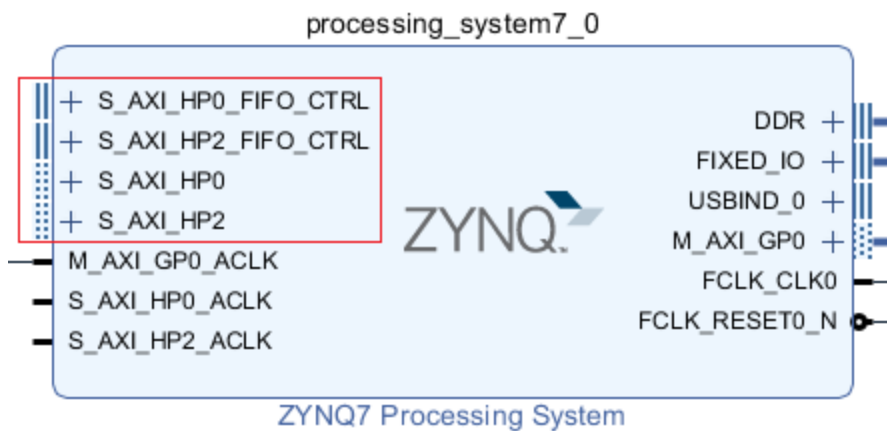
- Double click the Zynq PS block to open the customization settings
- Go to the PS-PL Configuration, expand HP Slave AXI Interface and enable S_AXI_HP0 and S_AXI_HP1



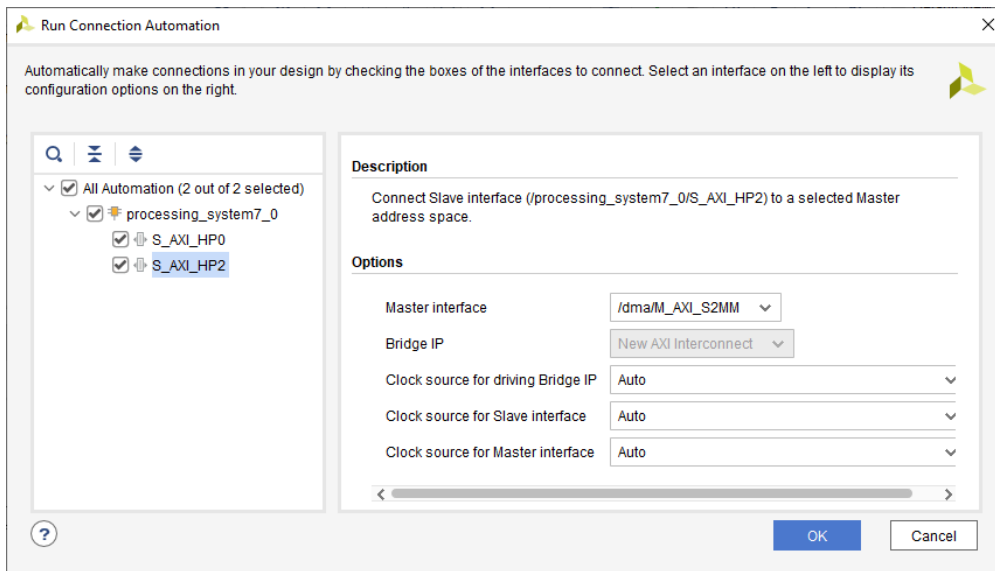
You can expand the S AXI HP ports and check the data width is set to 64. Remember, these data width settings are configured at boot time must match the size that was specified in your PYNQ image. This 64 by default for PYNQ images.

- Click OK to conform the changes

Notice the HP ports are enabled and *Run Connection Automation* is available again.



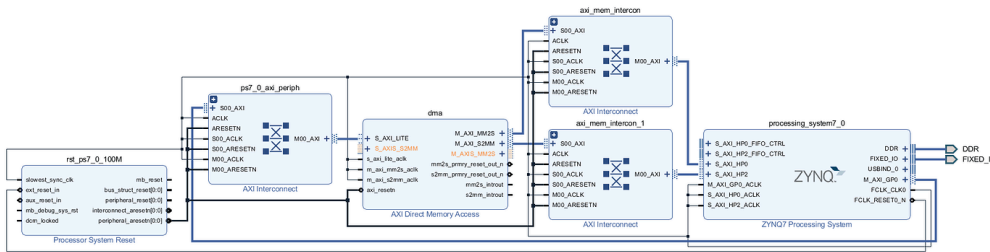
- Click on *Run connection automation* again
- Select *S_AXI_HP0* and for the *Master Interface* select */dma/M_AXI_MM2S*
- Select *S_AXI_HP2* and this time select */dma/M_AXI_S2MM* for the *Master Interface*



It doesn't matter which HP port a DMA master is connected to.

- Click **OK** to accept the changes

The Block Design should now look like this:



Only the DMA AXI Stream ports remain unconnected.

AXI Stream Ports

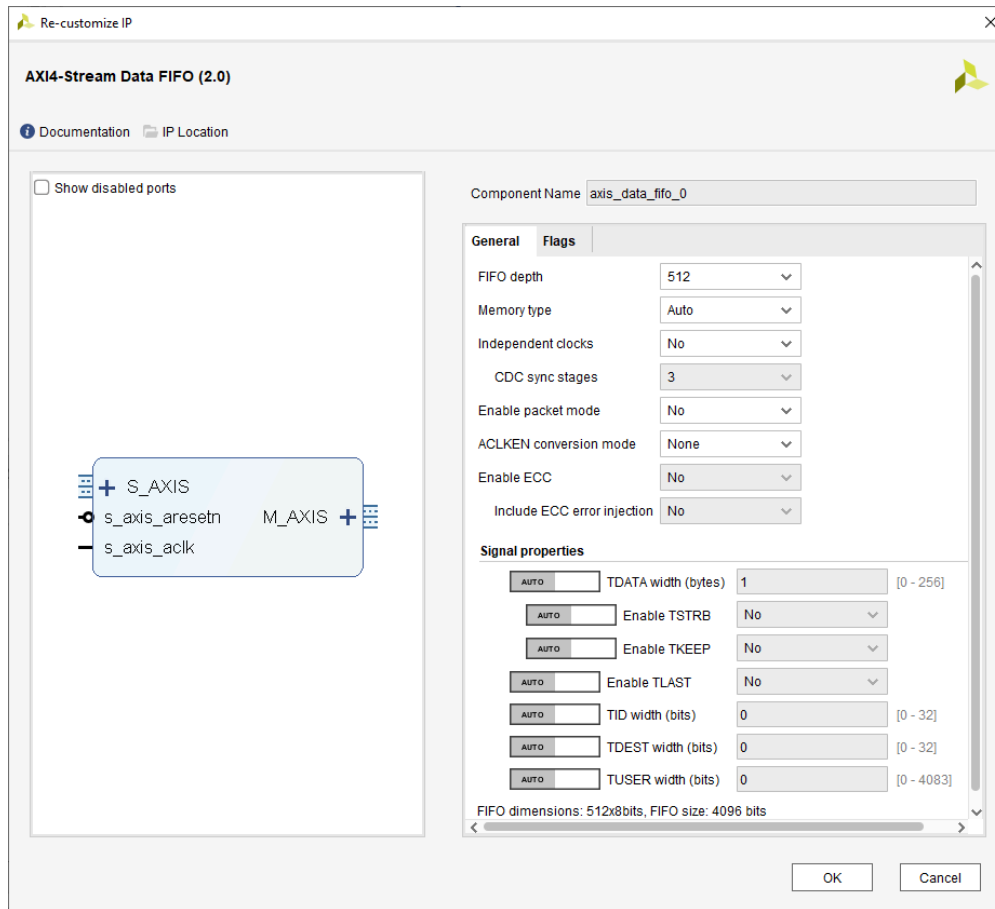
In this example I'm going to connect the AXI Stream ports in a loopback configuration. I could connect the ports directly to each other, but instead I will add an AXI Stream FIFO just to add some IP into the data path. In a real design you could replace the FIFO with your IP. In the next tutorial I will show how to add a HLS IP with AXI Stream ports to your design and use it with the DMA. For now we will add the FIFO:

- Add the **AXI4-Stream Data FIFO** to the design

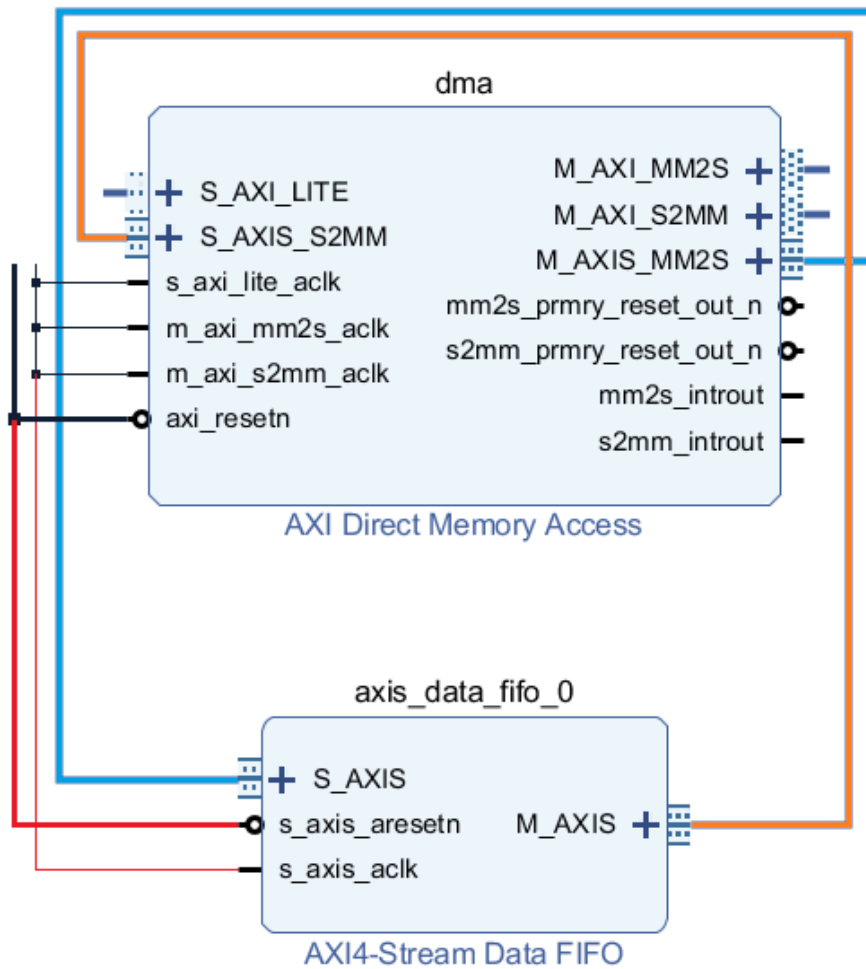
There are several different FIFO IP blocks in the Vivado IP catalog. Make sure to select the correct one.

We will use the default settings for the FIFO. Note that the AXI Stream sidebands or signal properties are set to *AUTO*. Some AXI signals are optional, and your design can function without them. The **TLAST** signal, which is part of the AXI standard, is required for the DMA to work properly. When creating your own IP, especially HLS IP, you need to make sure this signal is included. This can be a common problem when working with the DMA. I'll cover this in more detail in a later tutorial.

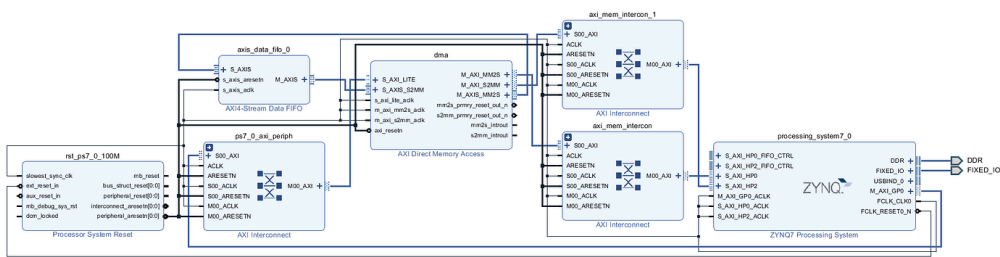
For this design Vivado will automatically infer the correct *sideband signals* for the AXI Stream interface to the FIFO.



- Make the following connections:
 - DMA: M_AXIS_MM2S → axis_data_fifo_0: S_AXIS
 - DMA: S_AXIS_S2MM → axis_data_fifo_0: M_AXIS
 - axis_data_fifo_0: s_axis_aclk to the Zynq PS FCLK_CLK0 (every clock port in this design is connected to this clock)
 - axis_data_fifo_0: s_axis_aresetn to the peripheral_aresetn on the *Processor System Reset* block



The design is now complete.



- Press F6 to run design validation and make sure there are no errors.

There are no external pins used in this design so no additional constraints are needed.

- Generate the HDL wrapper and generate the bitstream

You will need to the .bit file and the .hwh file for this design. Create a new folder for this example on your PYNQ board, and copy the BIT file and the HWH file to this directory.

The [second part of this tutorial](#) will show how to use the DMA hardware design from PYNQ.

If you have questions about this post, or problems getting it to work in your own design, please make a new post in the support forum.

13 Likes

[Tutorial: PYNQ DMA \(Part 2: Using the DMA from PYNQ\)](#)

[Tutorial: using a HLS stream IP with DMA \(Part 1: HLS design\)](#)

[How to use the DMA](#)

[Cyclic DMA Mode](#)

[Need help to use cdma in PYNQ Z2](#)

[Using PYNQ for DMA over PCIe on UltraScale+ MPSoC](#)

[Ultra96-V2 DMA performance](#)

[DMA channel not started](#)

[VIVADO block port design question](#)

[Transfer image through DMA](#)

[Phase jump with ADC acquisition on RFSOC board](#)

[Sending data to custom IP and receiving it back](#)

[DMA with custom IP written in verilog](#)

[Developing Verilog AXI Stream Overlays and interfacing to ZYNQ](#)

[Mismatch of PYNQ Data Stream from DMA on Ultrascale+](#)

[DMA channel not started on RFSoc 4x2](#)

[Could not find IP or hierarchy LSTM_Top_0 in overlay](#)

[Send and receive vector of data via DMA](#)

[Sobel filter \(Vitis vision library\) implementation on Pynq-Z board](#)

[DMA for float array](#)

[Operate on 2D-Array](#)

[Help Needed: Connecting PYNQ Z1 with Arduino/Raspberry Pi for OFDM Data Transmission](#)

[VIVADO block port design question](#)

[Board hangs when accessing AXI DMA from Python during RF loopback bring-up](#)

[Sharing buffer address with programmable logic](#)

[RuntimeError:"DMA does not support unaligned transfers"](#)

[Using USB port in PYNQ-Z2](#)

[PYNQ DFX Partial Reconfiguration: DMA Fails or Returns Garbage After Loading Partial Bitstreams](#)

[Communication PYNQ Z2 to PC via USB \(UART IP\)](#)

[Building a daq from RFSoc 4x2?](#)

[Tutorial: AXI Master interfaces with HLS IP](#)

[Using USB port in PYNQ-Z2](#)

[Added IP before HDMI out IP not working](#)

[PYNQ-ZU - Could not find IP or hierarchy xfft in overlay](#)

[Tutorial: PYNQ DMA \(Part 2: Using the DMA from PYNQ\)](#)

[DMA Stuck on recvchannel.wait\(\)](#)

[AXI DMA puzzle \(maybe\)](#)

[Sobel filter \(Vitis vision library\) implementation on Pynq-Z board](#)

[RFSoc Single DAC](#)

[Accesing RAM from PL only](#)

[How to connect ethernet port to PL](#)

[Designing an Overlay using Vivado Integrated Logic Analyzer \(ILA\) \[Part 1\]](#)

[cathalmccabe](#) Split this topic 2 December 6, 2021, 9:58am

3 posts were split to a new topic: [PYNQ DMA tutorial on ZU+](#)

[ggillett](#) 3 December 14, 2021, 5:12pm

Hi [@cathalmccabe](#) ,

Useful tutorial.

I don't understand why a 26 bit wide counter only leads to 8 MB DMA transfers or a 14 bit one leads to 2k.

Are these typos or am I missing something?

Cheers,

Geoff.

1 Like

[mizan](#) 4 December 15, 2021, 6:18am

$2^{14} = 16384 \text{ bit} = 2\text{KB}$

$2^{26} = 67108864 \text{ bit} = 8\text{MB}$

1 Like

[ggillett](#) 5 December 15, 2021, 12:57pm

OK then the question is why is it counting bits? DMA is bitwise? The RAM is byte-wise addressable at best.

From PG288 pg71

“The number of bytes is equal to $2^{*}[\text{Length Width}]$. So a Length Width of 26 gives a byte count of 67,108,863 bytes.”

2 Likes

[mizan](#) 6 December 16, 2021, 5:56am

 ggillett:

“The number of bytes is equal to $2^{*}[\text{Length Width}]$. So a Length Width of 26 gives a byte count of 67,108,863 bytes.”

So, obviously, my understanding was wrong. Maybe each combination is responsible for 1 byte of data (i am just assuming). Might be the software driver for maximum btt is 8MB for contiguous memory, but don't explain about 2KB at 14bit buffer length register.

1 Like

[cathalmccabe](#) 7 December 21, 2021, 12:57pm

Apologies, Scatter Gather is 8MB. It should be 2^{26} or 67,108,863 bytes for the simple DMA controller. Updated in the main article.

https://www.xilinx.com/support/documentation/ip_documentation/axi_cdma/v4_1/pg034-axi-cdma.pdf

Cathal

3 Likes

[Deontae](#) 8 December 23, 2021, 11:56am

Nice tutorial i am trying it it will be helpful if you post more parts about it in future thank you.

[UPSers Login](#)

1 Like

[kbhopi94](#) 9 June 21, 2022, 6:58pm

Hi,

I am working on Zynq Ultrascale+ RFSoc2X2 board. I can see there is no Zynq PS IP core provided for this board. I would really appreciate if you can let me know what modifications/alternatives can be implemented in this design for that board?

1 Like

[cathalmccabe](#) 10 June 23, 2022, 1:14pm

For the RFSoc 2x2 which is a Zynq UltraScale+ RFSoc the PS block is: "Zynq UltraScale+ MPSoC"
See the base overlay for the 2x2 as an example.

[rfsoc_2x2_base_overlay.pdf](#)

328.49 KB

The DMA IP will be similar.

The configuration of the PS block will be different - you have more options and more interfaces you can use. You can enable and connect the DMA to the HP in the FPD (Full Power Domain).

If this is your first time working with Zynq Ultrascale+ devices, you may want to try a beginner tutorial first.

Cathal

3 Likes

[pradeep_gautam](#) 12 July 14, 2022, 11:17am

Deontae:

ing it it will be helpful if you po

HI ,

I am using the AXI-DMA on ZCU111 board with PYNQ. I am using the scatter gather mode and allocating number of buffer.

I am not able to allocate more that few MBs of total memory(8192 buffers X 8192 bytes for each buff).

It there any restriction on how much memory can be allocated in a single Jupyter notebook?

1 Like

[fpgaenthusiast](#) 13 August 19, 2022, 2:36pm

Hello, I am wondering how we can connect our own IP with the DMA blocks?

[briansune](#) 14 August 19, 2022, 3:03pm

Hello,

You can find this tutorial useful:

[Data is not write back from DMA](#)

Hi Nagendra, It would be easy to test out this with such tutorial as it is reading back accordingly to you question. Or a small adjustment can also make to fit your application. All detail settings and setups are show in the link. [[- YouTube](#)]

1 Like

[cathalmccabe](#) 15 August 19, 2022, 4:20pm

Sorry for delay replying. The PYNQ buffers are contiguous. The amount of memory you can allocate will be determined by the amount of memory you have on the board, and the amount of that memory that can be allocated contiguously.

Cathal

1 Like

[cathalmccabe](#) 16 August 19, 2022, 4:22pm

The FIFO in the design is intended as a loopback test. The DMA has AXI stream interfaces, so you need corresponding interfaces on your IP or an adapter to convert from AXI stream to your IP.

Cathal

2 Likes

[cathalmccabe](#) 19 August 22, 2022, 1:33pm

The simplest way to connect this would be to use a 256 bit wide DMA interface, and ignore the upper 256-136 = 120 bits. This would not be a very efficient use of memory but would be easy to connect.

You would need to add some logic to connect your IP to the AXI stream interface.

To do this more efficiently you need to figure out how you store the data in memory that you want to transfer to your IP. The memory is 32 bit, or the interface between the PS and PL is 64 bit. If you are going to transfer 1x 136 bits at a time you could read 32/64 bits sequentially via the DMA, “deserialise” or reconstruct them into your 136 bit type (ignoring the upper bits).

For example, if you read 32 bits at a time, you need 5 x 4 bytes (160bits), and transfer these in parallel to your IP. In this case your IP would be running 5x slower than the DMA, as it takes at least 5 cycles to get the data that you transfer to your IP in one cycle. (If you use 64 bits, you would adjust the clocks accordingly.)

If you want to transfer a stream of 136 bits, again you need to determine how you store these in memory, transfer 32/64 bits at a time and deserialise into your 136 bit type. You could potentially store 136 bits contiguously (i.e. not byte aligned) and boost the efficiency of the memory transfer, but the logic to unpack your 136 bits will be more complicated.

Cathal

1 Like

[fpgaenthusiast](#) 20 August 22, 2022, 1:46pm

Thank you for the reply. I am new to this so is there any tutorial regarding the logic I need to connect the IP to the AXI stream interface?

[fpgaenthusiast](#) 21 August 23, 2022, 3:38pm

Is there any existing converter (raw data to AXI stream and the other way around) or does this need to be done by myself?

[fpgaenthusiast](#) 22 August 24, 2022, 12:41pm

Any idea [@cathalmccabe](#) ?

[ggillett](#) 23 August 24, 2022, 2:19pm

I think you'll need to add some handling of the AXIS handshake (TREADY/TVALID) signals to your SPI RTL module. Maybe also TLAST which indicates the end of the DMA packet. I don't think you will find any existing IP that you can simply plug in.

2 Likes

[cathalmccabe](#) 24 August 24, 2022, 2:36pm

Yes, as [@ggillett](#) says, you need to do this yourself. AXI stream protocol has a handshake, so you need to decide how you want to manage this. You may be able to do something really simple like connect your data to the AXI stream data, if your "start" signal acts like a valid, you could add some logic for the handshake/valid.

There are some good tutorials on the internet. E.g. AXI stream:

This is for an AXI slave, and is a little dated, but may still be of use:

[Creating a custom IP block in Vivado](#)

Update 2017-11-01: Here's a newer tutorial on creating a custom IP with AXI-Streaming interfaces
Tutorial Overview In this tutorial we'll create a custom AXI IP block in Vivado and modify its functionality by integrating custom VHDL code. We'll be...

Cathal

2 Likes

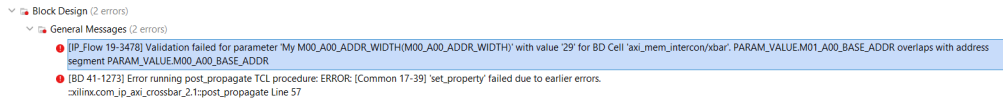
[fpgaenthusiast](#) 25 August 24, 2022, 3:19pm

I will take a look. Thank you so much!

1 Like

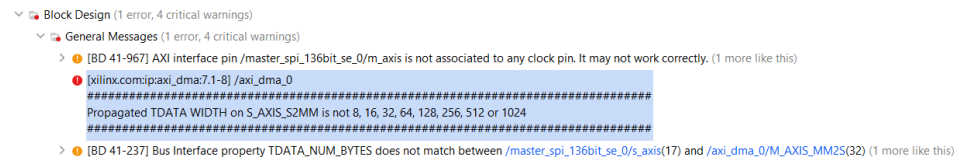
[fpgaenthusiast](#) 26 August 26, 2022, 9:00am

When I try to validate the design, I am getting the following error:

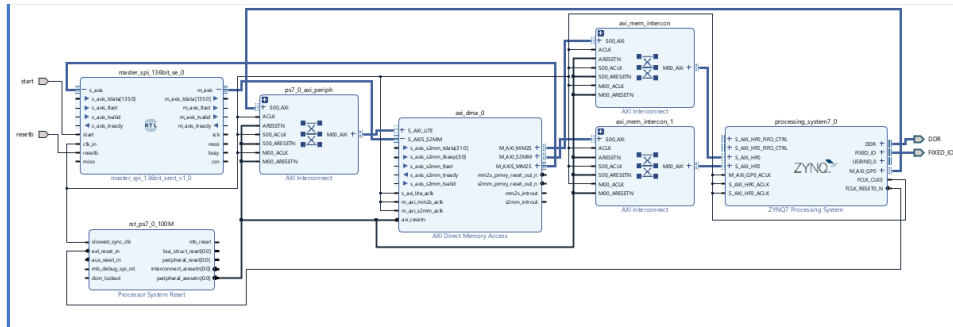


[fpgaenthusiast](#) 27 August 2022, 9:00am

I managed to make an AXI stream interface for my IP, however when I try to connect it using a 256 bit DMA interface I am getting this error.



This is how my block design is connected:



1 Like

[cathalmccabe](#) 28 August 30, 2022, 11:29am

The DMA only supports the widths mentioned in the error. You could expand your signal using a “concat” block and concatenate zeros.

You would ignore the upper bits.

There will some inefficiency, but may be good enough for your use case.

Regards,

Cathal

1 Like

[VIVADO block port design question](#)

[VIVADO block port design question](#)

[cathalmccabe](#) Split this topic 29 September 14, 2022, 1:51pm

A post was split to a new topic: [Add multiple DMAs](#)

[cathalmccabe](#) Split this topic 30 September 19, 2022, 4:28pm

A post was split to a new topic: [How to use BRAM from PS](#)

[willcbrn](#) 31 September 26, 2023, 6:01pm

How did you figure this issue out? I'm getting the same error.

1 Like

[cathalmccabe](#) Split this topic 32 December 22, 2023, 8:01am

A post was split to a new topic: [Axi dma zcu111](#)

[Axi dma zcu111](#)

[RFSoc 4x2 How to generate square wave](#)

[marioruiz](#) Split this topic 33 June 18, 2024, 7:53am

A post was split to a new topic: [DMA channel not started on RFSoc 4x2](#)

[Jean-Michel](#) 34 November 15, 2024, 2:23pm

Hi Cathal,

In the tutorial you strongly advise not to allow unaligned transfers because that this is not supported by PYNQ v2.6.

However the code of the dma library in PYNQ seems to handle the case where the dre is enabled.

Could you just give me a bit more detailed explanation about this statement ?

I'm currently using PYNQ 2.7 but it seems that the PYNQ dma library code did not change that much.

Thank you in advance,

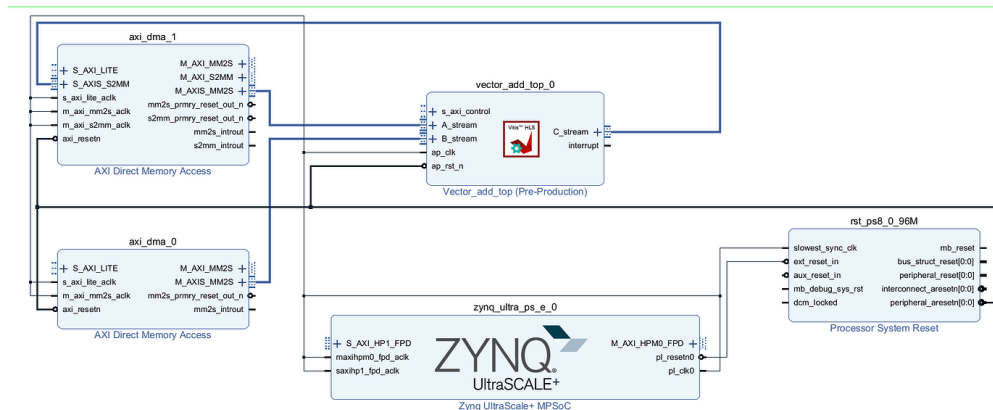
Jean-Michel

1 Like

[Tritone7-6](#) 36 March 7, 2025, 7:43am

This is such a really great tutorial! I had some doubts when using the KR 260 to connect to the DMA. As shown in the screenshot, I set the Stream Data Width of all the DMA modules to 32. How can I set the Data Width of the PS part AIX interface?

For the Master axi interface on PS, it needs to connect three s_axi (from DMA 0, DMA 1 and IP) for control. The Slave axi interface on the PS is similar (it connects to MM2S for DMA 0, S2MM and MM2S for DMA 1



).

Should the AXI on the PS also be set to a 32-bit Data Width in this case? Or should it be set to 128? I am a little confused about the concept of Data Width matching due to the AXI Interconnect module.

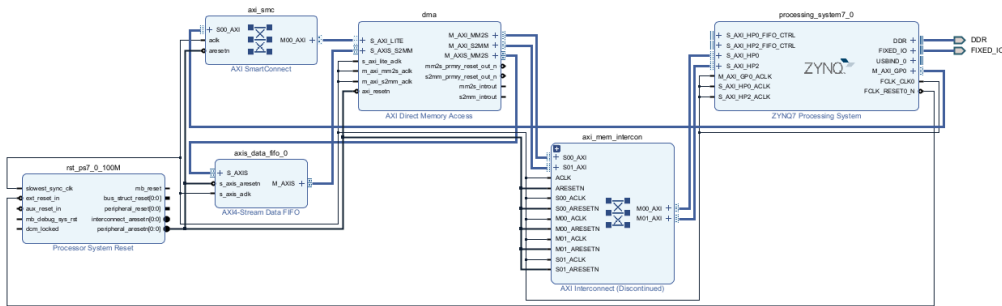
1 Like

rgbblue 37 April 1, 2025, 5:25pm

Hi, i get the following critical warning when trying to validate the design:

[BD 41-1356] Slave segment </processing_system7_0/S_AXI_HP2/HP2_DDR_LOWOCM> is not assigned into address space </dma/Data_S2MM>. Please use Address Editor to either assign or exclude it.

Should i be concerned. I am also using Vivado 2024.2 - the results from running connection automation are not the same as in the tutorial:



1 Like

marioruiz 38 April 2, 2025, 7:15am

Hi @rgbblue ,

Should i be concerned

Yes, you will not be able to use MMIO. The warning has advise on how to solve this. **Please use Address Editor to either assign or exclude it.**

Vivado 2024.2 is not tested neither verified by the PYNQ team. We suggest you use Vivado 2022.1.

Mario

1 Like

hypothermic 39 September 28, 2025, 10:31pm

Thank you for the tutorial, it was easy to follow.

One small remark:

Make the following connections:

- ...
- axis_data_fifo_0: s_axis_aclk to the Zynq PS FCLK_CLK0 (every clock port in this design is connected to this clock)
- axis_data_fifo_0: s_axis_aclk to the peripheral_aresetn on the *Processor System Reset* block

In the last bullet point: `s_axis_aclk` should be `s_axis_aresetn`

1 Like

[ffb-LANL](#) 40 October 11, 2025, 12:08am

Thank you for the detailed example. We are trying to record long continuous ADC waveforms into PL RAM. The design is based on the RFSOC-MTS example where AXI DMA is used to fill the PL RAM with an ADC stream. The PL RAM size in the design is 4G, but it appears that the ADC waveform length is limited by the size of the DMA buffer length register and is much smaller than the size of the PL RAM. Is there any way to overcome this limit and record streams as large as 4G?

Related topics

Topic	Replies	Activity
Tutorial: PYNQ DMA (Part 2: Using the DMA from PYNQ)	4	January 10, 2023
AXI DMA puzzle (maybe)	11	August 15, 2025
FIFO using PYNQ DMA	7	July 10, 2020
AXI DMA not working on PYNQ-Z1	11	December 16, 2019
Load data to IP via DMA	22	February 14, 2020

